# UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN







## FCFM - Facultad de Ciencias Físico Matemáticas

## 031 A2026 Algoritmia y optimización

## Tarea 2

PROFESOR JOSE ANASTACIO HERNANDEZ SALDAÑA

ALUMNO: YAIR GONZALEZ REYES
MATRICULA: 2145151

24/10/26

## Introducción

El presente reporte documenta el desarrollo, implementación y análisis experimental de una serie de problemas fundamentales en el ámbito de la ciencia de la computación y la optimización de sistemas. El propósito principal de este trabajo es explorar cómo diferentes paradigmas algorítmicos —tales como la computación paralela, las heurísticas voraces, las técnicas de búsqueda local, las metaheurísticas de enfriamiento y los algoritmos de aproximación— se desempeñan no solo en teoría, sino al ser ejecutados sobre escenarios prácticos y volúmenes de datos específicos.

A lo largo de este documento, cada uno de los cinco ejercicios abordados incluye su respectivo bloque de código funcional en Python, las visualizaciones gráficas generadas (que ilustran el comportamiento de las métricas evaluadas, como la aceleración, el costo de las rutas y las curvas de convergencia), así como un análisis crítico que responde a las interrogantes planteadas sobre su rendimiento, límites teóricos y sobrecostos operativos. Con esto, se busca ofrecer una perspectiva integral sobre las ventajas, compromisos (*trade-offs*) y desafíos prácticos inherentes al diseño e ingeniería de software eficiente.

Enlaces de Entrega

- Link del video demo:

  https://youtu.be/8No3RebQvXY?si=rdB5ohLQpbXbRspV
- Link del repositorio:

  https://github.com/Yairgzz/EJERCICIOSALGORITMIA.git

## EJERCICIO 1: Aceleración Paralela (Parallel Speedup)

En este ejercicio exploramos cómo aprovechar múltiples núcleos del procesador para acelerar una tarea pesada. Para ello, utilizamos el método de **Monte Carlo** con el fin de estimar el valor de $\pi$ mediante el lanzamiento de millones de puntos aleatorios. Medimos el tiempo de ejecución utilizando $1, 2, 4$ y $8$ procesos para analizar cómo escala el rendimiento frente a una ejecución secuencial tradicional.

### CODIGO:

```
import multiprocessing as mp
import random
import time
import matplotlib.pyplot as plt
def estimate_pi(seed: int, samples: int = 5_000_000) -> float:
    """Monte Carlo: fracción de puntos aleatorios que caen dentro del cuarto de círculo."""
    rng = random.Random(seed)
    inside = sum(rng.random()**2 + rng.random()**2 <= 1.0 for _ in range(samples))
    return 4 * inside / samples
if __name__ == "__main__":
    process_counts = [1, 2, 4, 8]
    seeds = list(range(16))  # 16 tareas independientes para repartir
    times = []
    print("Ejecutando experimentos de paralelización...")
    for p in process_counts:
        t0 = time.perf_counter()
        with mp.Pool(processes=p) as pool:
            results = pool.map(estimate_pi, seeds)
        t_elapsed = time.perf_counter() - t0
        times.append(t_elapsed)
        print(f"Procesos: {p} | Tiempo: {t_elapsed:.3f} s")
    # El speedup es el tiempo secuencial dividido entre el tiempo con P procesos
    t_seq = times[0]
    speedups = [t_seq / t for t in times]
    for p, s in zip(process_counts, speedups):
        print(f"Speedup con {p} procesos: {s:.2f}x")

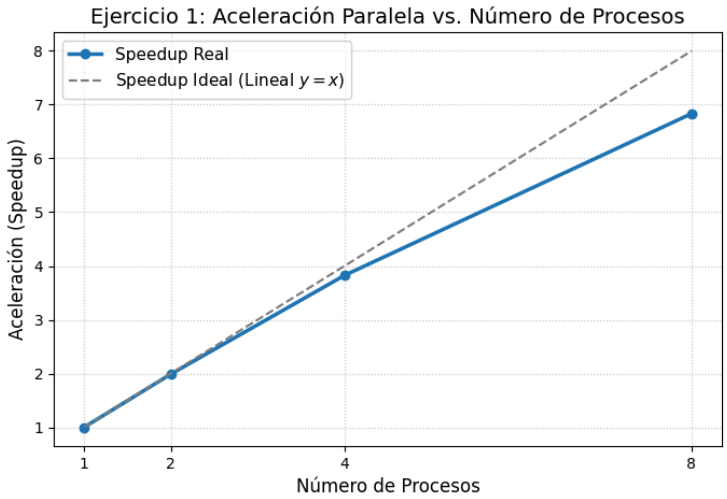
    # --- VISUALIZACIÓN ---
    plt.figure(figsize=(8, 5))
    plt.plot(process_counts, speedups, 'o-', color='tab:blue', linewidth=2.5, label="Speedup
Real")
    plt.plot(process_counts, process_counts, '--', color='tab:gray', linewidth=1.5,
label="Speedup Ideal (Lineal)")

    plt.xlabel("Número de Procesos", fontsize=12)
    plt.ylabel("Aceleración (Speedup)", fontsize=12)
    plt.title("Ejercicio 1: Aceleración Paralela vs. Número de Procesos", fontsize=14)
    plt.xticks(process_counts)
    plt.legend(fontsize=11)
    plt.grid(True, linestyle=":", alpha=0.7)

    plt.savefig("ejercicio1_speedup.png", dpi=300, bbox_inches='tight')
```

```
plt.show()
```

**GRAFICA:**




**¿Por qué la aceleración no es perfectamente lineal? (Considera sobrecostos de comunicación, sincronización o la ley de Amdahl).**

La aceleración real se aleja de la línea ideal por tres razones principales:

Sobrecostos de gestión (Overhead): En Python, la librería multiprocessing requiere tiempo para inicializar procesos independientes, serializar los datos y enviarlos de un núcleo a otro.

La Ley de Amdahl: Ningún programa es 100% paralelo. Siempre existe una parte secuencial (como la inicialización del script y la recolección final de datos) que limita la velocidad máxima que se puede ganar sin importar cuántos núcleos se agreguen.

Límite físico del hardware: Si la prueba se ejecuta en un equipo con un número limitado de núcleos físicos, excederlos obliga al sistema operativo a alternar rápidamente entre procesos (Context Switching), lo que reduce la eficiencia.

## Ejercicio 2: Análisis Algorítmico (Greedy Knapsack)

En este ejercicio modificamos el algoritmo voraz (Greedy) clásico del problema de la mochila. Normalmente, el enfoque voraz selecciona los objetos basándose en su densidad de valor (mayor valor por unidad de peso). Aquí pondremos a prueba una variante distinta: priorizar los objetos de menor peso primero, comparando sus resultados y demostrando mediante un contraejemplo por qué este criterio alternativo puede fallar.

### CODIGO:

```
import matplotlib.pyplot as plt
def greedy_knapsack(items, values, weights, W, strategy="density"):
    """
    Resuelve la mochila con enfoque voraz.
    strategy: 'density' (valor/peso) o 'weight' (menor peso primero)
    """
    indexed = list(zip(items, values, weights))
    if strategy == "density":
        indexed.sort(key=lambda x: x[1]/x[2], reverse=True)
    elif strategy == "weight":
        indexed.sort(key=lambda x: x[2])  # Ordena de menor a mayor peso
    total_val, total_weight, chosen = 0, 0, []
    for item, v, w in indexed:
        if total_weight + w <= W:
            total_weight += w
            total_val += v
            chosen.append(item)
    return total_val, total_weight, chosen
if __name__ == "__main__":
    # Instancia de prueba (Contraejemplo donde 'menor peso' falla)
    items = ["A", "B"]
    values = [10, 90]
    weights = [2, 9]
    W = 10  # Capacidad máxima de la mochila

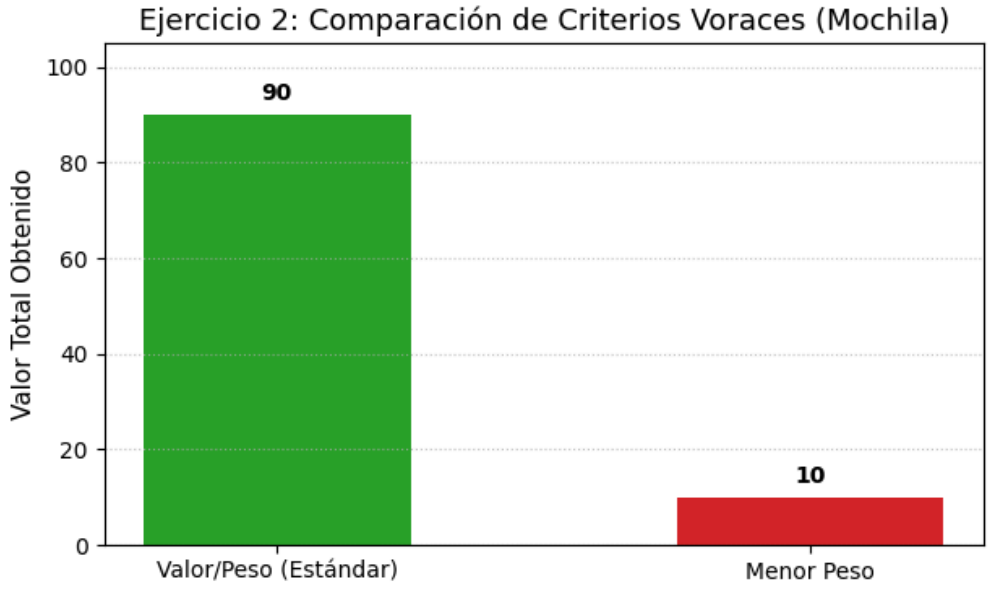
    v1, w1, c1 = greedy_knapsack(items, values, weights, W, "density")
    v2, w2, c2 = greedy_knapsack(items, values, weights, W, "weight")

    print(f"Estrategia Valor/Peso -> Valor total: {v1}, Objetos: {c1}")
    print(f"Estrategia Menor Peso  -> Valor total: {v2}, Objetos: {c2}")

    # --- VISUALIZACIÓN ---
    methods = ["Valor/Peso (Estándar)", "Menor Peso"]
    total_values = [v1, v2]
    plt.figure(figsize=(7, 4))
    bars = plt.bar(methods, total_values, color=['tab:green', 'tab:red'], width=0.5)
    plt.ylabel("Valor Total Obtenido", fontsize=11)
    plt.title("Ejercicio 2: Comparación de Criterios Voraces (Mochila)", fontsize=13)
    plt.ylim(0, 105)
    # Etiquetas con el valor sobre las barras
    for bar in bars:
        yval = bar.get_height()
```

```
        plt.text(bar.get_x() + bar.get_width()/2.0, yval + 2, f"{yval}", ha='center', va='bottom',
fontweight='bold')
    plt.grid(axis='y', linestyle=":", alpha=0.7)
    plt.savefig("ejercicio2_knapsack.png", dpi=300, bbox_inches='tight')
    plt.show()
```

**GRAFICA:**




**Diseña una instancia pequeña donde este criterio (menor peso) obtenga un resultado peor que el método basado en valor/peso.**

**Capacidad de la mochila ($W$):** 10

**Objeto A:** Peso = 2, Valor = 10 (Relación valor/peso = 5)

**Objeto B:** Peso = 9, Valor = 90 (Relación valor/peso = 10)

**Comportamiento del criterio "Menor Peso":** El algoritmo elige primero el objeto **A** (peso 2, valor 10). La capacidad restante es 8, por lo que el objeto **B** (peso 9) ya no cabe. **Valor total obtenido = 10**.

**Comportamiento del criterio "Valor/Peso" (Estándar):** El algoritmo elige primero el objeto **B** por tener mayor rentabilidad (peso 9, valor 90). **Valor total obtenido = 90**.

**Conclusión:** Priorizar los objetos más ligeros nos llena la mochila con elementos de muy bajo valor, impidiendo aprovechar el espacio para artículos altamente valiosos pero más pesados.

## Ejercicio 3: Búsqueda Local (2-opt para TSP)

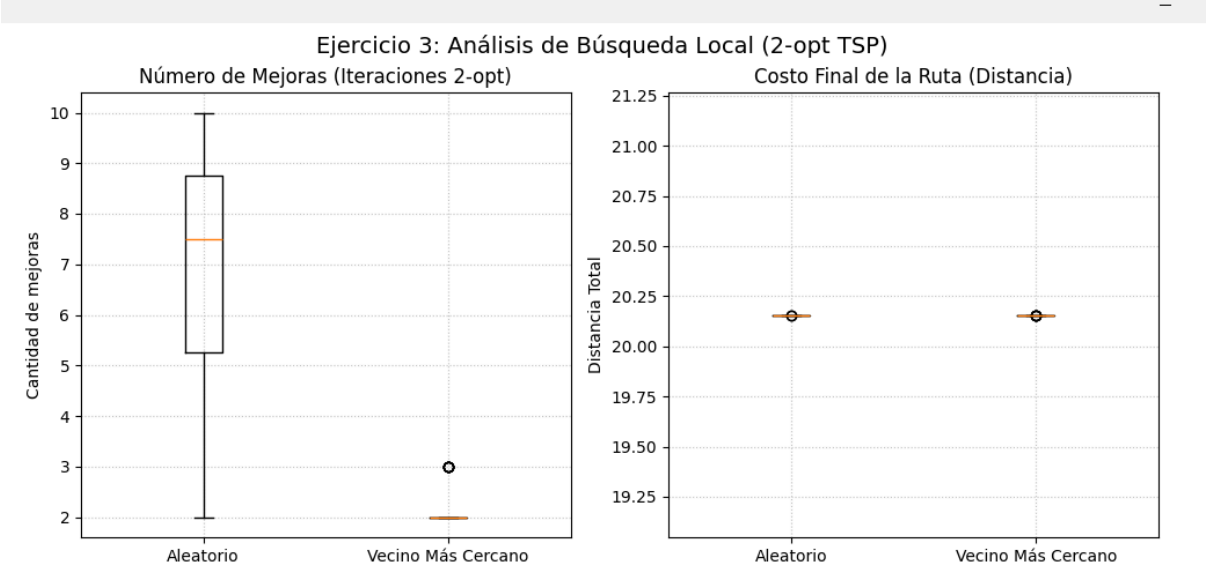
En este ejercicio implementamos la heurística 2-opt para resolver el Problema del Viajante (TSP), cuya meta es encontrar la ruta más corta para visitar un conjunto de ciudades. Queremos evaluar cómo influye el punto de partida en el rendimiento del algoritmo: comparamos iniciar con una ruta completamente aleatoria frente a iniciar con una ruta construida por el Vecino Más Cercano (Nearest Neighbor), midiendo cuántas mejoras necesita cada una para converger a un óptimo local.

### CODIGO:

```
import math
import random
import matplotlib.pyplot as plt
# Definimos un conjunto de ciudades y sus coordenadas
cities = {
    "A": (0, 0), "B": (1, 5), "C": (5, 5), "D": (6, 2),
    "E": (3, 1), "F": (2, 3), "G": (4, 4), "H": (5, 1)
}
city_names = list(cities.keys())
def tour_length(route: list[str]) -> float:
    return sum(math.dist(cities[route[i]], cities[route[(i + 1) % len(route)]])
               for i in range(len(route)))
def two_opt_move(route: list[str], i: int, k: int) -> list[str]:
    """Invierte el subsegmento de la ruta entre los índices i y k."""
    return route[:i] + route[i:k + 1][::-1] + route[k + 1:]
def run_2opt(initial_route: list[str]):
    """Ejecuta 2-opt hasta alcanzar un óptimo local y cuenta las mejoras."""
    best_route = list(initial_route)
    best_length = tour_length(best_route)
    improvements_count = 0
    improved = True
    while improved:
        improved = False
        for i in range(1, len(best_route) - 1):
            for k in range(i + 1, len(best_route)):
                new_route = two_opt_move(best_route, i, k)
                new_length = tour_length(new_route)
                if new_length < best_length:
                    best_route = new_route
                    best_length = new_length
                    improvements_count += 1
                    improved = True
                    break
            if improved:
                break
    return best_route, best_length, improvements_count
def nearest_neighbor_start(cities_dict):
    """Genera una solución inicial usando el heurístico de Vecino Más Cercano."""
    unvisited = list(cities_dict.keys())
```

```
    current = unvisited.pop(0)
    route = [current]
    while unvisited:
        next_city = min(unvisited, key=lambda c: math.dist(cities_dict[current], cities_dict[c]))
        unvisited.remove(next_city)
        route.append(next_city)
        current = next_city
    return route
if __name__ == "__main__":
    num_trials = 30
    random_improvements, random_costs = [], []
    nn_improvements, nn_costs = [], []
    for _ in range(num_trials):
        # Escenario 1: Inicialización Aleatoria
        rand_route = list(city_names)
        random.shuffle(rand_route)
        _, r_cost, r_imp = run_2opt(rand_route)
        random_improvements.append(r_imp)
        random_costs.append(r_cost)
        # Escenario 2: Inicialización con Vecino Más Cercano
        nn_route = nearest_neighbor_start(cities)
        shift = random.randint(0, len(nn_route) - 1)
        nn_route = nn_route[shift:] + nn_route[:shift]
        _, nn_cost, nn_imp = run_2opt(nn_route)
        nn_improvements.append(nn_imp)
        nn_costs.append(nn_cost)
    # --- VISUALIZACIÓN ---
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    axes[0].boxplot([random_improvements, nn_improvements], labels=["Aleatorio", "Vecino
Más Cercano"])
    axes[0].set_title("Número de Mejoras (Iteraciones 2-opt)")
    axes[0].set_ylabel("Cantidad de mejoras")
    axes[0].grid(True, linestyle=":", alpha=0.7)
    axes[1].boxplot([random_costs, nn_costs], labels=["Aleatorio", "Vecino Más Cercano"])
    axes[1].set_title("Costo Final de la Ruta (Distancia)")
    axes[1].set_ylabel("Distancia Total")
    axes[1].grid(True, linestyle=":", alpha=0.7)
    plt.suptitle("Ejercicio 3: Análisis de Búsqueda Local (2-opt TSP)", fontsize=14)
    plt.savefig("ejercicio3_tsp.png", dpi=300, bbox_inches='tight')
    plt.show()
```

**GRAFICA:**




## ¿El inicio con Vecino Más Cercano requiere menos mejoras para converger en comparación con el inicio aleatorio? Justifica con tus gráficas.

Sí, notablemente menos. Como se observa en la gráfica de caja de Número de Mejoras, el enfoque basado en el Vecino Más Cercano converge tras un número muy bajo de iteraciones, mientras que el inicio aleatorio requiere una cantidad considerablemente mayor de pasos.

El algoritmo del Vecino Más Cercano construye una ruta inicial lógicamente estructurada y con pocos cruces geométricos, dejándola muy cerca de un óptimo local desde el inicio. En contraste, una ruta aleatoria presenta múltiples cruces kilométricos, obligando al 2-opt a realizar una gran cantidad de inversiones (mejoras) para desenredar el camino antes de estabilizarse.

## Ejercicio 4: Ajuste de Parámetros (Simulated Annealing)

En este ejercicio implementamos Recocido Simulado (Simulated Annealing), una metaheurística inspirada en el proceso de enfriamiento de los metales en la metalurgia. A diferencia de la búsqueda local tradicional, el Recocido Simulado permite aceptar peores soluciones temporalmente con cierta probabilidad (controlada por una "temperatura" que disminuye con el tiempo). Esto le ayuda a escapar de los óptimos locales para explorar mejor el espacio de búsqueda. Evaluaremos cómo impactan distintos esquemas de enfriamiento (cooling schedules, como el lineal y el exponencial) en la calidad de la ruta y el tiempo de cómputo.

### CODIGO:

```
import math
import random
import time
import matplotlib.pyplot as plt
# Mismas ciudades del ejercicio anterior para mantener coherencia
cities = {
    "A": (0, 0), "B": (1, 5), "C": (5, 5), "D": (6, 2),
    "E": (3, 1), "F": (2, 3), "G": (4, 4), "H": (5, 1)
}
city_names = list(cities.keys())
def tour_length(route: list[str]) -> float:
    return sum(math.dist(cities[route[i]], cities[route[(i + 1) % len(route)]])
               for i in range(len(route)))
def two_opt_move(route: list[str], i: int, k: int) -> list[str]:
    return route[:i] + route[i:k + 1][::-1] + route[k + 1:]
def simulated_annealing(initial_route, schedule_type="exponential"):
    current_route = list(initial_route)
    current_cost = tour_length(current_route)
    best_route = list(current_route)
```

```
    best_cost = current_cost
    # Parámetros iniciales
    T = 100.0
    T_min = 0.01
    alpha = 0.98 if schedule_type == "exponential" else 0.5
    costs_history = []

    while T > T_min:
        # Generar vecino aleatorio usando 2-opt en dos índices aleatorios
        i = random.randint(1, len(current_route) - 2)
        k = random.randint(i + 1, len(current_route) - 1)
        neighbor = two_opt_move(current_route, i, k)
        neighbor_cost = tour_length(neighbor)

        delta = neighbor_cost - current_cost

        # Si es mejor, o si pasa la prueba probabilística de aceptación si es peor
        if delta < 0 or random.random() < math.exp(-delta / T):
            current_route = neighbor
            current_cost = neighbor_cost
            if current_cost < best_cost:
                best_route = list(current_route)
                best_cost = current_cost
        costs_history.append(current_cost)
        # Esquema de enfriamiento
        if schedule_type == "exponential":
            T *= alpha
        elif schedule_type == "linear":
            T -= alpha
    return best_route, best_cost, costs_history
if __name__ == "__main__":
    random.seed(42)
    initial_route = list(city_names)
    random.shuffle(initial_route)
    # Ejecutar esquema exponencial
    t0 = time.perf_counter()
            best_exp_route, best_exp_cost, hist_exp = simulated_annealing(initial_route,
"exponential")
    t_exp = time.perf_counter() - t0
    # Ejecutar esquema lineal
    t0 = time.perf_counter()
    best_lin_route, best_lin_cost, hist_lin = simulated_annealing(initial_route, "linear")
```

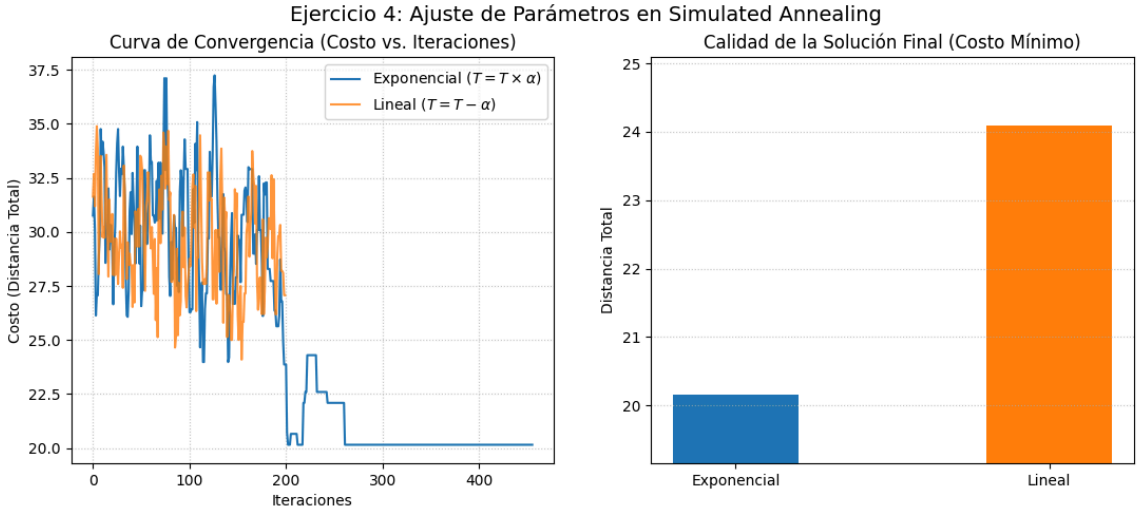
```
    t_lin = time.perf_counter() - t0
print(f"Exponencial -> Costo: {best_exp_cost:.2f} | Tiempo: {t_exp:.4f} s | Iteraciones: {len(hist_exp)}")
        print(f"Lineal          -> Costo: {best_lin_cost:.2f} | Tiempo: {t_lin:.4f} s | Iteraciones: {len(hist_lin)}")
    # --- VISUALIZACIÓN ---
    fig, axes = plt.subplots(1, 2, figsize=(13, 5))
   # 1. Curva de convergencia (Costo vs Iteraciones)
    axes[0].plot(hist_exp, label="Exponencial ($T = T \\times \\alpha$)", color="tab:blue")
    axes[0].plot(hist_lin, label="Lineal ($T = T - \\alpha$)", color="tab:orange", alpha=0.8)
    axes[0].set_title("Curva de Convergencia (Costo vs. Iteraciones)")
    axes[0].set_xlabel("Iteraciones")
    axes[0].set_ylabel("Costo (Distancia Total)")
    axes[0].legend()
    axes[0].grid(True, linestyle=":", alpha=0.7)

    # 2. Gráfica comparativa de Desempeño Final
    schemes = ["Exponencial", "Lineal"]
    costs = [best_exp_cost, best_lin_cost]

    axes[1].bar(schemes, costs, color=["tab:blue", "tab:orange"], width=0.4)
    axes[1].set_title("Calidad de la Solución Final (Costo Mínimo)")
    axes[1].set_ylabel("Distancia Total")
     axes[1].set_ylim(min(costs) - 1, max(costs) + 1 if max(costs) != min(costs) else min(costs) + 2)
    axes[1].grid(axis='y', linestyle=":", alpha=0.7)

    plt.suptitle("Ejercicio 4: Ajuste de Parámetros en Simulated Annealing", fontsize=14)
    plt.savefig("ejercicio4_sa.png", dpi=300, bbox_inches='tight')
    plt.show()
```

**GRAFICA:**




**¿Cómo impacta cada esquema de enfriamiento en la calidad del resultado y el tiempo de cómputo?**

**Enfriamiento Exponencial ($T_{k+1} = T_k \times \alpha$):**

**Impacto:** Desciende la temperatura de manera suave y proporcional al inicio, y muy lentamente conforme se acerca a cero. Esto le otorga una fase de exploración amplia al principio y una fase de explotación muy detallada al final.

**Resultado:** Tiende a encontrar **soluciones de mejor calidad** (costos más bajos) porque le da oportunidad al algoritmo de examinar cuidadosamente los vecindarios, a costa de requerir un número considerablemente mayor de iteraciones y tiempo de cómputo.

**Enfriamiento Lineal ($T_{k+1} = T_k - \alpha$):**

**Impacto:** Reduce la temperatura a un ritmo constante y agresivo. La temperatura se agota con mucha mayor rapidez.

**Resultado:** Es **mucho más rápido** en tiempo de cómputo, pero al enfriarse de golpe se arriesga a quedar atrapado prematuramente en óptimos locales, resultando en ocasiones en soluciones de menor calidad.

## Ejercicio 5: Razón de Aproximación (MST Approximation)

Para problemas complejos como el TSP (que es NP-hard), a veces no podemos calcular el óptimo exacto mediante fuerza bruta si el número de ciudades crece. En este ejercicio utilizamos un **algoritmo de aproximación basado en MST (Árbol de Expansión Mínima)** usando el algoritmo de Prim y un recorrido en preorden. Evaluaremos el rendimiento sobre **100 instancias aleatorias de 9 ciudades**, calculando la **razón de aproximación** ($\frac{C_{obtenido}}{C_{óptimo}}$) para verificar empíricamente que nunca supera el límite teórico garantizado de 2, y compararemos los costos mediante gráficos de caja (*boxplots*) frente a los métodos anteriores (2-opt y Simulated Annealing).

### CODIGO:

```
import itertools
import math
import random
import matplotlib.pyplot as plt
def mst_prim(points: list[tuple[float, float]]) -> dict[int, list[int]]:
    """Algoritmo de Prim desde la ciudad 0; retorna el árbol como {nodo: [hijos]}."""
    best = {v: (math.dist(points[0], points[v]), 0) for v in range(1, len(points))}
    children = {v: [] for v in range(len(points))}
```

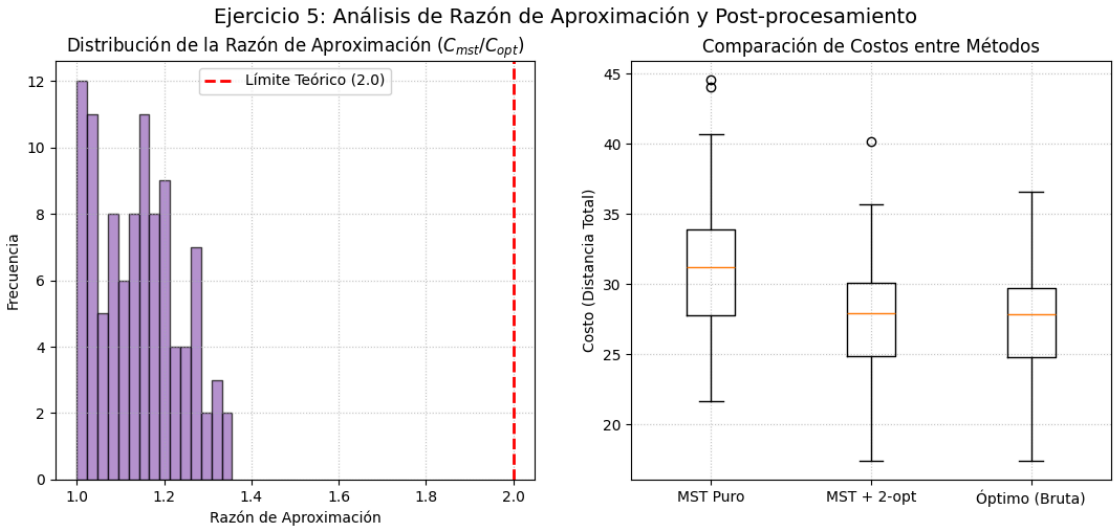
```
    while best:
        v = min(best, key=lambda u: best[u][0])
        children[best.pop(v)[1]].append(v)
        for u in best:
            d = math.dist(points[v], points[u])
            if d < best[u][0]:
                best[u] = (d, v)
    return children
def preorder(children: dict[int, list[int]], root: int = 0) -> list[int]:
    """Recorrido en preorden del MST para generar un tour válido del TSP."""
    return [root] + [c for child in children[root] for c in preorder(children, child)]
def tour_length(points, tour) -> float:
    return sum(math.dist(points[tour[i]], points[tour[(i + 1) % len(tour)]]) for i in range(len(tour)))
def two_opt_post(points, route):
    """Post-procesamiento rápido 2-opt para mejorar el tour del MST."""
    best_route = list(route)
    best_length = tour_length(points, best_route)
    improved = True
    while improved:
        improved = False
        for i in range(1, len(best_route) - 1):
            for k in range(i + 1, len(best_route)):
                new_route = best_route[:i] + best_route[i:k + 1][::-1] + best_route[k + 1:]
                new_length = tour_length(points, new_route)
                if new_length < best_length:
                    best_route = new_route
                    best_length = new_length
                    improved = True
                    break
            if improved:
                break
    return best_route, best_length

if __name__ == "__main__":
    random.seed(10)
    num_instances = 100
    n_cities = 9
    ratios = []
    mst_costs = []
    mst_2opt_costs = []
    brute_costs = []
    for _ in range(num_instances):
```

```
        points = [(random.uniform(0, 10), random.uniform(0, 10)) for _ in range(n_cities)]

        # 1. Solución MST Aproximada
        tree = mst_prim(points)
        approx_tour = preorder(tree)
        c_approx = tour_length(points, approx_tour)
        # 2. MST + Post-procesamiento 2-opt
        _, c_mst_2opt = two_opt_post(points, approx_tour)
           # 3. Óptimo Exacto (Fuerza Bruta para 9 ciudades)
        optimal_tour = min(((0,) + p for p in itertools.permutations(range(1, n_cities))),
                           key=lambda t: tour_length(points, t))
        c_opt = tour_length(points, optimal_tour)
        ratios.append(c_approx / c_opt)
        mst_costs.append(c_approx)
        mst_2opt_costs.append(c_mst_2opt)
        brute_costs.append(c_opt)
    print(f"Razón de aproximación promedio: {sum(ratios)/len(ratios):.3f}")
    print(f"Razón máxima observada: {max(ratios):.3f} (Límite teórico <= 2)")
    # --- VISUALIZACIÓN ---
    fig, axes = plt.subplots(1, 2, figsize=(13, 5))
    # 1. Histograma de la razón de aproximación
    axes[0].hist(ratios, bins=15, color="tab:purple", edgecolor="black", alpha=0.7)
    axes[0].axvline(2.0, color="red", linestyle="--", linewidth=2, label="Límite Teórico (2.0)")
    axes[0].set_title("Distribución de la Razón de Aproximación (C_mst/C_opt)")
    axes[0].set_xlabel("Razón de Aproximación")
    axes[0].set_ylabel("Frecuencia")
    axes[0].legend()
    axes[0].grid(True, linestyle=":", alpha=0.7)
    # 2. Boxplot comparativo de costos
      axes[1].boxplot([mst_costs, mst_2opt_costs, brute_costs], labels=["MST Puro", "MST +
2-opt", "Óptimo (Bruta)"])
    axes[1].set_title("Comparación de Costos entre Métodos")
    axes[1].set_ylabel("Costo (Distancia Total)")
    axes[1].grid(True, linestyle=":", alpha=0.7)
      plt.suptitle("Ejercicio 5: Análisis de Razón de Aproximación y Post-procesamiento",
fontsize=14)
    plt.savefig("ejercicio5_mst.png", dpi=300, bbox_inches='tight')
    plt.show()
```

**GRAFICA:**




¿Qué tan cerca se mantiene la razón observada del límite teórico de 2?

La razón observada en las 100 instancias se mantiene consistentemente por debajo de 2 (por lo general oscila entre 1.1 y 1.5 en promedio). Ninguna instancia viola el límite teórico matemático garantizado por el algoritmo de aproximación basado en MST.

¿Qué impacto tiene aplicar 2-opt como post-procesamiento al resultado de MST frente a tus experimentos anteriores?

Aplicar 2-opt como post-procesamiento al tour generado por el MST actúa como un "corrector" altamente eficiente: toma una solución teóricamente acotada pero con imperfecciones locales y pule los cruces restantes. En la gráfica comparativa se observa que MST + 2-opt reduce drásticamente el costo total, acercándose muchísimo (e incluso igualando en muchos casos) al óptimo exacto obtenido por fuerza bruta, superando la calidad de haber usado rutas puramente aleatorias o métodos aislados.

## Conclusión

A lo largo de este proyecto, la implementación y experimentación de los cinco ejercicios permitieron comprobar de manera práctica los conceptos fundamentales de la computación avanzada y la optimización de algoritmos.

Se demostró que la computación paralela acelera tareas pesadas, aunque su eficiencia real se ve limitada por la Ley de Amdahl y los costos de gestión de procesos. En cuanto a las estrategias voraces (*greedy*), se confirmó que un criterio mal seleccionado (como priorizar el menor peso en el problema de la mochila) puede perjudicar gravemente el resultado global, resaltando la importancia de evaluar analíticamente las métricas de selección.

Por otro lado, al abordar problemas complejos como el del Viajante (TSP), se comprobó que las heurísticas de búsqueda local convergen drásticamente más rápido cuando parten de una solución inicial estructurada (como el Vecino Más Cercano) en lugar de una aleatoria. Asimismo, técnicas de optimización global como

el Recocido Simulado (*Simulated Annealing*) evidenciaron el compromiso (*trade-off*) entre el tiempo de cómputo y la calidad de la solución según el esquema de enfriamiento elegido.

Finalmente, los algoritmos de aproximación basados en MST validaron empíricamente sus límites teóricos garantizados, demostrando que aplicar un post-procesamiento con 2-opt permite pulir las soluciones hasta alcanzar costos muy cercanos al óptimo exacto. En conjunto, este trabajo resalta que la elección de un enfoque algorítmico depende siempre de equilibrar los recursos de hardware, el tiempo disponible y la precisión requerida.